From ingress-nginx to Traefik

Giovanni Toraldo

Hyland

About me

- DevOps Engineer at Hyland
- Open Source and tech enthusiast
- Reenactor during the weekends
- More about me on gionn.net

Ops readiness team

My team ensures that Alfresco can be deployed and operated reliably in multiple environments. We focus on:

- Helm charts and GitOps best practices
- Docker images and Compose for local development
- CI/CD pipelines and automation
- Performance testing and monitoring 🆕

Open source projects we maintain

- Containerized deployments:
  - acs-deployment umbrella Helm chart for the whole stack and compose files for local development
  - alfresco-helm-charts: component-level charts for more flexibility
  - alfresco-dockerfiles-bakery: build your own Docker images with customizations
- Classic deployments:
  - alfresco-ansible: Ansible playbooks for on-premises/VM deployments

Table of contents

1. The Problem: ingress-nginx deprecation
2. Traefik on Kubernetes
3. KinD Improvements
4. Live Demo
5. Traefik on Docker Compose
6. ActiveMQ Authentication (ACS 26+)
7. Summary

The Problem: ingress-nginx deprecation

Part 1

What is an Ingress Controller?

An **Ingress Controller** is a Kubernetes component that manages external HTTP/HTTPS access to services inside a cluster.

It watches for `Ingress` resources and sets up a reverse proxy accordingly.

```
Internet → LoadBalancer → Ingress Controller → Services → Pods
```

- Routes traffic based on hostnames and paths
- Handles TLS termination
- Can enforce rate limits, auth, redirects, …

> Without an Ingress Controller installed, `Ingress` resources have no effect.

ingress-nginx has been retired

The Kubernetes project officially announced the **retirement of ingress-nginx** in November 2025.

After March 2026:

- No new releases or feature updates
- No security patches
- No bug fixes

> "Users are encouraged to migrate to a maintained ingress controller."
> – kubernetes.io/blog, November 2025

Ingress is not going away

The `Ingress` API is still part of Kubernetes and will continue to be supported.

The `Gateway` API is also emerging as a more powerful and flexible alternative, but adoption is still in early stages.

The key point is that **the deprecation only affects the ingress-nginx controller** – not the Ingress API itself – a lot of people are missing this nuance.

Why it matters for Alfresco

`acs-deployment` has used **ingress-nginx as the default** Ingress Controller for Helm deployments since day one.

Staying on ingress-nginx means:

- Running **unpatched** software after the EOL date
- Increasing **security risk** over time
- Falling behind on Kubernetes compatibility

We need a modern, maintained replacement – and we chose **Traefik**.

Traefik on Kubernetes

Part 2

Why Traefik?

Traefik is a modern, cloud-native reverse proxy and Ingress Controller actively maintained by Traefik Labs.

- Supports standard Kubernetes `Ingress` resources (and its own CRD `IngressRoute`)
- Built-in dashboard, metrics, tracing, access logs
- Actively developed, well-documented, widely adopted
- Provides a compatibility mode for ingress-nginx annotations

Our charts use **standard `Ingress` resources** plus a few nginx-specific annotations. Switching to Traefik doesn't require helm chart changes.

Installing Traefik

```
helm repo add traefik https://traefik.github.io/charts
helm repo update
helm upgrade --install traefik traefik/traefik \
  --set providers.kubernetesIngressNginx.enabled=true \
  --set logs.access.enabled=true \
  --namespace traefik --create-namespace
```

- Enabling `kubernetesIngressNginx` provider tells Traefik to watch `Ingress` resources with `ingressClassName: nginx` and provides compatibility with existing ingress-nginx annotations.

Charts Backward Compatibility

Our latest charts provides `global.ingressClassName: nginx` as default – **existing deployments continue to work** with the current `Ingress` resources.

Optional fine-grained control via component chart values:

- `.ingress.className` : override the IngressClass name
- `.ingress.annotations` : add more controller-specific annotations
- `.ingress.enabled: false` : disable bundled Ingress to bring your own

The nginx ingressClass must still be present

It's critical to keep the `nginx` `IngressClass` resource in the cluster, even after uninstalling ingress-nginx, otherwise Traefik won't pick up any `Ingress` resources.

```yaml
apiVersion: networking.k8s.io/v1
kind: IngressClass
metadata:
  name: nginx
spec:
  controller: k8s.io/ingress-nginx
```

Migrating an existing cluster

Zero-downtime strategy from the official Traefik guide:

Steps (diagram on next slide)

1. Install Traefik **alongside** NGINX (both serve traffic)
2. Verify Traefik handles your Ingresses via its own LoadBalancer IP (e.g. edit your `/etc/hosts` to point to Traefik's IP)
3. Add Traefik IP to DNS – progressive traffic shift
4. Remove NGINX IP from DNS, wait for propagation
5. Uninstall NGINX - delete admission webhook if installed without Helm
6. Ensure `nginx` `IngressClass` is still present

Upgrade strategy

```
Before:
  DNS → NGINX LB → Services

After Traefik installed:
  DNS → NGINX LB   → Services
      → Traefik LB → Services

After DNS shift:
  DNS → Traefik LB → Services
        NGINX LB   → Services (draining)

After NGINX uninstall:
  DNS → Traefik LB → Services
```

KinD Improvements

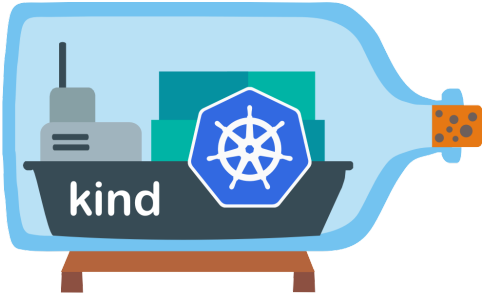
Part 3

What is KinD?

KinD is a tool for running Kubernetes clusters locally using just a Docker runtime.

It's widely used for local development and testing on the CI.

Successor of Minikube or any other native local Kubernetes solution (e.g. Rancher Desktop, Docker Desktop Kubernetes).



The old ingress-nginx on KinD

Running ingress-nginx on KinD required a custom cluster configuration exposing host HTTP/HTTPS ports:

```
cat <<EOF | kind create cluster --config=-
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
nodes:
- role: control-plane
  kubeadmConfigPatches:
  - |
    kind: InitConfiguration
    nodeRegistration:
      kubeletExtraArgs:
        node-labels: "ingress-ready=true"
  extraPortMappings:
  - containerPort: 80
```

Problems with the old approach

A few issues with this setup:

- Ports 80 and 443 must be **free on the host**
- Requires a **patched ingress-nginx manifest** for KinD
- Cluster config is tightly coupled to the ingress controller
- Not really representative of a real Kubernetes environment

cloud-provider-kind

cloud-provider-kind is a plugin that brings **native** `LoadBalancer` **support** to KinD clusters.

It monitors `Service` resources of type `LoadBalancer` and automatically assigns them an IP address – just like a real cloud provider would.

```
brew install cloud-provider-kind

# Run in a separate terminal while the cluster is up:
sudo cloud-provider-kind
```

Benefits of cloud-provider-kind

- No special KinD cluster config required
- Works with any standard Ingress Controller
- On Podman/Docker Desktop (macOS/Windows): automatically enables `--enable-lb-port-mapping` for `localhost` access
- Ingress testing experience is **much closer to a real cluster**

New KinD setup

Four simple steps – no custom cluster config needed:

```
# 1. Create a plain KinD cluster
kind create cluster

# 2. Run cloud-provider-kind (keep running in background)
sudo cloud-provider-kind

# 3. Install Traefik (standard Helm install)
helm upgrade --install traefik traefik/traefik \
  --set providers.kubernetesIngressNginx.enabled=true \
  --namespace traefik --create-namespace

# 4. Verify LoadBalancer IP assigned and Traefik is ready
kubectl get svc -n traefik
```

Live Demo

Part 4

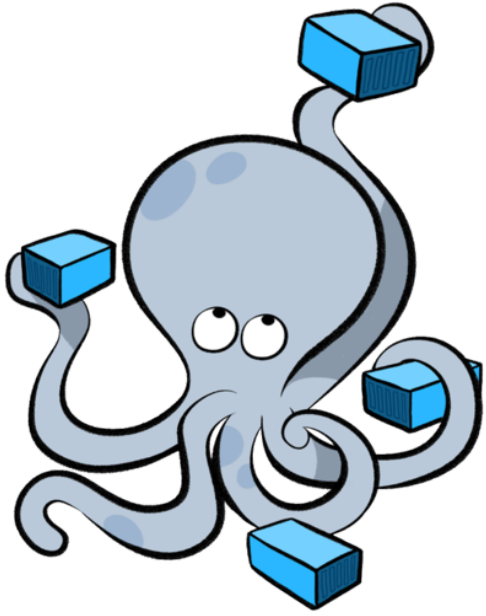
Traefik on Docker Compose

Part 5

Traefik in Compose

While the Helm migration happened in March 2026, **Traefik was already the proxy in our Docker Compose bundles** for both Community and Enterprise since late 2024.

A single reverse proxy across all environments → simpler mental model, unified configuration patterns.



How Traefik works with Compose

- No separate config file.
- Routing is driven by **Docker labels** on each service.
  - Labels are **evaluated live** by Traefik - no need to restart the proxy, just the service being routed.
  - Configure every aspect of routing, middleware, limit, TLS, etc. via labels

```yaml
services:
  alfresco:
    image: quay.io/alfresco/alfresco-content-repository:26.1.0
    labels:
      - "traefik.http.routers.alfresco.rule=PathPrefix(`/`)"
```

Compose with Traefik: example

```yaml
services:
  alfresco:
    image: quay.io/alfresco/alfresco-content-repository:26.1.0
    labels:
      - "traefik.enable=true"
      - "traefik.http.routers.alfresco.rule=PathPrefix(`/`)"
      - "traefik.http.middlewares.limit.buffering.maxRequestBodyBytes=5368709120"
      - "traefik.http.routers.alfresco.middlewares=limit@docker"

  proxy:
    image: traefik:v3
    command:
      - "--providers.docker=true"
      - "--entrypoints.web.address=:8080"
      - "--entrypoints.web.transport.respondingTimeouts.readTimeout=1200"
```

Compose `extends` feature

We now leverage Docker Compose extends feature to improve maintainability.

```
docker-compose/
├── commons/                ← shared base definitions (Traefik labels, …)
├── compose.yaml            ← Enterprise which extends commons
├── community-compose.yaml ← Community which extends commons
├── 23.N-compose.yaml       ← older supported versions
└── solr6-overrides.yaml    ← optional overrides for specific components
```

Customizations should be extensions in `commons` rather than copy-pasting service definitions.

ActiveMQ Authentication (ACS 26+)

Part 6

ActiveMQ auth enabled by default

Starting with **ACS 26**, the `alfresco-activemq:6.2.x` image enables authentication by default. This applies to **both Compose and Helm**.

Affected services: `alfresco-repository`, `transform-router`, `transform-core-aio`, `alfresco-sync-service`, `search-enterprise`

Older `alfresco-activemq` images (pre-6.2.x) remain unauthenticated by default for backwards compatibility.

Configuring ActiveMQ credentials

On the broker (`alfresco-activemq` service / chart):

```
ACTIVEMQ_ADMIN_LOGIN: admin
ACTIVEMQ_ADMIN_PASSWORD: <your-password>
```

On every service that connects to ActiveMQ (or similar properties depending on the component):

```
SPRING_ACTIVEMQ_USER: admin
SPRING_ACTIVEMQ_PASSWORD: <your-password>
```

Summary

Part 7

What do you need to do?

Kubernetes / Helm

- Uninstall ingress-nginx, install Traefik with

  `providers.kubernetesIngressNginx.enabled=true`

- No changes to existing `Ingress` resources needed
- For existing clusters: follow the zero-downtime migration guide

What do you need to do?

KinD

- Drop the custom cluster config
- Use `kind create cluster` + `cloud-provider-kind`
- Install Traefik normally

What do you need to do?

Docker Compose

- Already on Traefik – nothing to do on the proxy side
- If on ACS 26+: add ActiveMQ credentials to all affected services
- Run compose files from within the `docker-compose/` directory

The bigger picture

This is not just a swap of one reverse proxy for another.

It's a **consistent modernization** across every Alfresco deployment surface:

- One mental model for routing configuration
- One set of documentation and debugging skills
- Actively maintained, cloud-native tooling
- Foundation for future improvements: Gateway API, advanced traffic management

Questions?

Reach me on LinkedIn: Giovanni Toraldo

or on my website gionn.net

Slides sources on GitHub: github.com/gionn/ingress-nginx-traefik